

부하가 늘수록 느려지던 이유

게이트웨이는 그냥 느린 것이 아니라, 바빠질수록 느려지고 있었습니다. 요청 하나의 비용이 "최근 처리한 요청 수"에 비례했기 때문입니다. 하드웨어를 늘려도 풀리지 않는 종류였습니다.

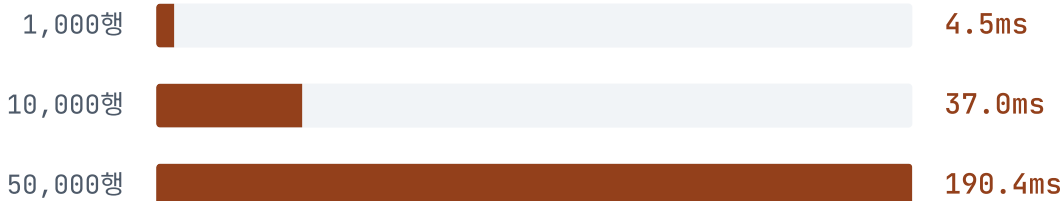
20×

15분 창에 20,000행이 쌓인 상태 · 요청당 81.9ms → 4.1ms · 단일 고루틴 처리량 12 → 246 req/s

무엇이 잘못돼 있었나

라우팅은 provider 건강도를 보고 대상을 고릅니다. 그 건강도를 계산하려면 `request_logs` 에서 최근 15분 안의 행을 전부 읽어 Go에서 집계해야 했고, 그 계산을 요청마다 2~3번 했습니다.

즉 요청 하나의 비용이 "최근 15분간 처리한 요청 수"에 비례했습니다. 그런데 그 수는 처리량에 비례합니다. 그래서 초당 작업량이 요청률의 제곱으로 늘고, 부하를 올릴수록 게이트웨이가 느려졌습니다.



15분 창 안의 REQUEST_LOGS 행 수에 따른 요청당 PROVIDER 건강도 비용 (수정 전)

50,000행은 초당 약 56요청으로 15분을 돌린 상태입니다. 그 지점에서 요청 하나가 다른 일을 시작하기도 전에 190ms를 건강도 계산에만 썼습니다.

왜 그동안 안 보였나

이 결함은 데이터가 없는 환경에서는 원리적으로 보이지 않습니다. 앞선 측정들이 이걸 놓친 이유가 그것입니다 — 테스트 DB에 행이 20개뿐이었고, 그때는 이 계산이 거의 공짜였습니다.

고친 방식과 실측

건강도를 캐시했습니다. 15분 창에서 3초는 데이터의 0.3%이므로 라우팅 결정을 바꾸지 못합니다. 부하와 무관하게 3초에 한 번만 계산합니다.

15분 창에 20,000행	요청당	처리량
수정 전	81.9 ms	12 req/s
수정 후	4.1 ms	246 req/s

수정 전 수치는 트래픽이 늘수록 계속 나빠지지만, 수정 후 수치는 그렇지 않습니다. 숫자보다 이 성질이 중요합니다.

이 캐시에는 무효화가 없습니다

다른 캐시들은 운영자가 쓴 값을 담기에 쓰기 시점에 지울 수 있습니다. 이건 트래픽에서 파생된 통계라 걸어들 수정 시점이 없습니다. TTL이 메커니즘 전부이고, 그래서 앞으로도 무효화를 붙이면 안 됩니다.

같은 부류의 두 번째 — 쿼터 사용량

쿼터는 "기간 사용량 vs 한도"로 집행되는데, 그 사용량을 요청마다 `request_logs` 와 `token_usage` 를 조인해 기간 전체에 걸쳐 집계했습니다. 월 단위 쿼터면 한 달치를 매 요청 다시 읽는 셈입니다.

스코프 (100,000행 · 월 기간)	기존 집계	일 단위 롤업	배수
global	63.1 ms	63 μs	1000×
team	106.8 ms	78 μs	1370×
ip	21.9 ms	46 μs	475×
api_key	11.9 ms	51 μs	233×

team 이 가장 나뉘던 것은 행마다 api_keys 상관 서브쿼리로 팀을 구했기 때문입 니다. 인덱스로는 풀리지 않았습니 다 — 복합 인덱스를 넣어도 api_key 는 12.1ms → 11.9ms로 그대로였고, 비용의 주범이 조인 자체였습니다.

해법은 요청을 기록하는 같은 트랜잭션에서 그날의 총계를 갱신하는 것입니다. 둘 이 어긋날 수 없고, 기록에 실패한 요청은 사용량도 남기지 않습니다. 보존 정책으 로 지워진 트래픽이 쿼터에서 빠지는 기존 동작은 그대로입니다.

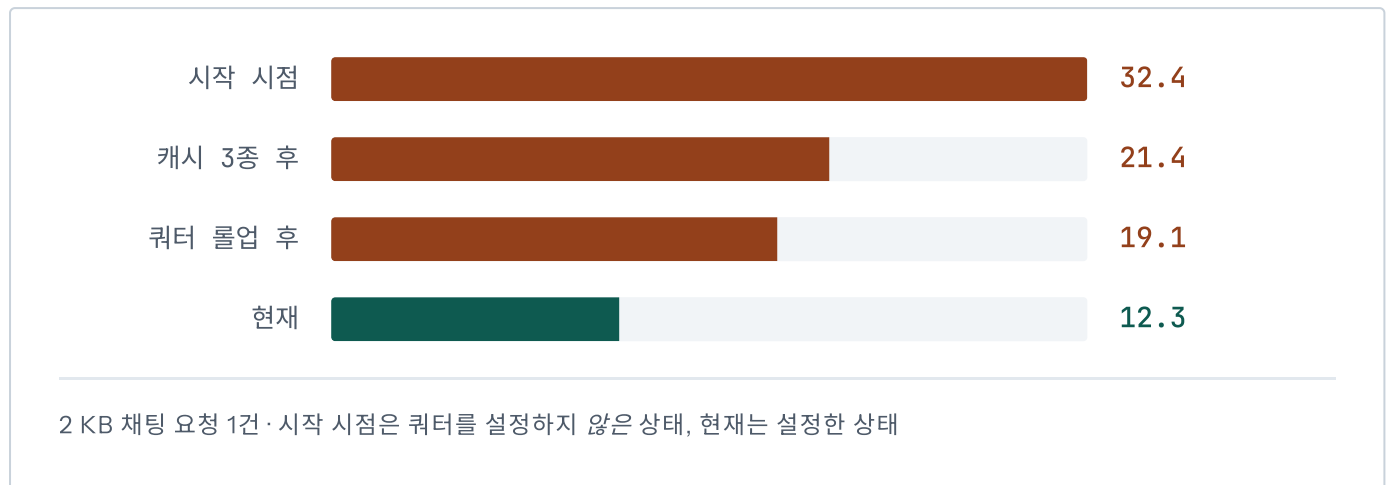
나머지 — 왕복 줄이기

위 둘이 구조적 문제였다면, 나머지는 같은 값을 한 요청 안에서 반복해서 읽던 것 들입니다. Postgres에서는 읽기 한 번이 네트워크 왕복 한 번입니다.

중복 읽기	1회 비용	요청당 횟수	절감
provider 설정	381 μs	5	1.90 ms
거버넌스 규칙	519 μs	3	1.56 ms
쿼터 정의	386 μs	3	1.16 ms
teams (인증)	~485 μs	1	0.49 ms
agent route	~400 μs	1	0.40 ms

쓰기 쪽도 줄였습니다. 한 요청이 여러 개 가질 수 있는 것들(프롬프트, 평가, 언어 통 계, 도구 호출)이 행마다 따로 쓰이고 있었습니다. 20턴 대화면 프롬프트만으로 왕복 20회입니다. 컬럼이 같으니 한 문장으로 묶었습니다 — llm_evaluations 는 요청당 6.55회에서 1.05회가 됐습니 다.

요청당 Postgres 문장 수



비교 조건이 현재 쪽에 불리하다는 점을 짚어둡니다. 32.4는 쿼터를 설정하지 않은 상태였고, 12.3은 설정한 상태입니다. 예전이라면 쿼터를 켜는 순간 훨씬 나빠졌을 조건입니다.

정상 상태에서 요청 경로가 DB에서 읽는 것은 이제 세 가지뿐입니다 — API 키, 그리고 설정된 쿼터별 사용량 총계. 나머지는 비동기 로거의 쓰기입니다.

요청 본문 처리도 함께 줄였습니다

큰 요청 하나를 처리하는 비용 자체도 손봤습니다. 1 MB 요청 기준입니다.

1 MB 요청	수정 전	수정 후
처리 시간	2.6 s	0.52 s
리닥션 대상 바이트	5.2 MB	1.04 MB
본문 대비 할당량	292×	99×

본문을 여러 번 리닥션하던 것을 한 번으로 줄이고, 모델명 하나 읽으려고 전체를 리닥션하던 경로를 없앴 결과입니다. 함께 요청 본문 크기 상한도 기본값을 넣었습니다(64 MiB) — 이전에는 상한이 없어 요청 하나가 게이트웨이 메모리를 다 쓸 수 있었습니다.

운영 환경에서 확인하는 법

- 인덱스 상태 — 관리자 콘솔 → 운영 홈 탭 맨 아래 카드, 또는 `GET /admin/index-health`. 선언한 스키마와 실제 DB의 차이, 그리고 접근 통계가 말해주는 추가·삭제 후보를 보여줍니다. 읽기 전용입니다.

- 부하 테스트 재현 — 이번 개선은 *이미 쌓인 트래픽*이 있어야 드러납니다. 빈 DB로 재면 개선 전후가 거의 같게 나옵니다. 15분 이상 부하를 걸어 창을 채운 뒤 비교하세요.
- 쿼터를 쓰신다면 그쪽 개선폭이 가장 큼니다. 업그레이드 직후 진행 중인 기간(일/월)은 예전 방식으로 계산되고, 다음 기간부터 빨라집니다.

측정에 대해 정직하게

어떤 수치를 믿어도 되고 어떤 수치는 참고만 해야 하는지 적어둡니다.

- 믿어도 되는 것 — 요청당 쿼리 수(로그로 직접 셈), 격리 측정한 함수별 비용, 그리고 20× 전후 비교(같은 박스·같은 실행에서 코드만 바뀌 측정).
- 참고만 — 개별 실행의 절대 wall-clock. 측정 박스(WSL2 + Docker Postgres)에서 같은 코드가 회당 11~26ms로 튀었습니다. 그래서 작은 개선은 end-to-end로 분해되지 않아, 해당 부분만 격리해 측정했습니다.
- 실제 환경 수치는 다릅니다 — 위 배수는 이 측정 조건에서의 값입니다. 방향과 성질(부하에 따라 나빠지던 것이 그러지 않게 됨)은 같지만, 배수는 데이터 양과 DB 성능에 따라 달라집니다.

v0.77.4 - v0.79.4 측정: SQLite / PostgreSQL 16 (Docker, WSL2)

요청 형태: 2 KB 채팅 완성 · 단일 고루틴